

**Báo cáo tổng kết training Data Engineer**  
**Xây dựng hệ thống phát hiện xâm nhập thời**  
**gian thực (Streaming IDS)**  
**Sử dụng Apache Kafka và Spark Structured Streaming**

|                 |   |                   |
|-----------------|---|-------------------|
| Họ và tên       | : | Lê Anh Quân       |
| Ngày tháng      | : | 28/08/2026        |
| Người hướng dẫn | : | Nguyễn Công Thịnh |

## LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến anh Vũ Xuân Tuyền - trưởng phòng, người đã tạo điều kiện để em được tham gia kỳ thực tập này. Em hiểu rằng nhận một sinh viên năm nhất, chưa có kinh nghiệm thực tế, không phải là một quyết định dễ dàng. Cơ hội đó là điểm khởi đầu cho toàn bộ những gì em trình bày trong báo cáo này.

Đặc biệt, em xin cảm ơn anh Nguyễn Công Thịnh, người đã trực tiếp hướng dẫn em trong suốt quá trình thực tập. Điều em học được nhiều nhất không phải là một công nghệ cụ thể, mà là cách tiếp cận vấn đề: tìm hiểu về mối liên kết giữa các bộ phận, và trình bày trung thực cả những kết quả không đạt như kỳ vọng ban đầu.

Cuối cùng, con xin cảm ơn mẹ - người đã tìm hiểu, chuẩn bị và nộp hồ sơ để con có được cơ hội này, và luôn tin rằng con làm được ngay cả khi con còn chưa chắc chắn về bản thân mình.

Do kiến thức và kinh nghiệm còn hạn chế, báo cáo chắc chắn không tránh khỏi thiếu sót. Em rất mong nhận được những góp ý để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

*Hà Nội, ngày 28 tháng 08 năm 2026*  
**Lê Anh Quân**

## Mục lục

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>I. Giới thiệu</b>                                         | <b>5</b>  |
| 1. Bối cảnh                                                  | 5         |
| 2. Vấn đề cần giải quyết                                     | 5         |
| 3. Mục tiêu và phạm vi                                       | 5         |
| 4. Những thay đổi so với đề cương                            | 6         |
| <b>II. Cơ sở lý thuyết</b>                                   | <b>7</b>  |
| 1. Hệ thống phát hiện xâm nhập                               | 7         |
| a. Phân loại theo vị trí triển khai                          | 7         |
| b. Phân loại theo phương pháp phát hiện                      | 8         |
| 2. Phát hiện dựa trên luật                                   | 8         |
| 3. Lý do chọn phương pháp rule-based                         | 9         |
| 4. Các chỉ số đánh giá                                       | 9         |
| 5. Bộ dữ liệu NSL-KDD                                        | 11        |
| 6. Kiến trúc phân tầng của một hệ thống IDS thực tế          | 11        |
| <b>III. Lựa chọn công nghệ</b>                               | <b>12</b> |
| 1. Tổng quan kiến trúc                                       | 12        |
| 2. Apache Kafka - Tầng tiếp nhận dữ liệu                     | 12        |
| 3. Apache Spark Structured Streaming - Tầng xử lý            | 13        |
| 4. ClickHouse - Tầng lưu trữ                                 | 13        |
| 5. Apache Superset - Tầng trực quan hóa                      | 14        |
| 6. Docker Compose - Tầng hạ tầng                             | 14        |
| <b>IV. Xây dựng hệ thống</b>                                 | <b>15</b> |
| 1. Cấu trúc mã nguồn                                         | 15        |
| 2. Bộ sinh dữ liệu                                           | 16        |
| 3. Hợp đồng lược đồ dữ liệu                                  | 16        |
| 4. Bộ máy luật - thiết kế không dùng UDF                     | 16        |
| 5. Thiết kế lưu trữ hai bảng                                 | 17        |
| a. Bảng detection                                            | 17        |
| b. Bảng traffic_counts                                       | 18        |
| 6. Xử lý bản ghi lỗi - phân loại hai nhóm                    | 18        |
| 7. Đo độ trễ                                                 | 18        |
| <b>V. Các vấn đề gặp phải và giải pháp</b>                   | <b>19</b> |
| 1. Truy vấn luồng bị treo do Catalyst constraint propagation | 19        |
| 2. Bộ đếm bản ghi lỗi luôn bằng 0                            | 21        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| 3. Tràn bộ nhớ do micro-batch không giới hạn                   | 23        |
| 4. ClickHouse chặn truy cập mạng của người dùng không mật khẩu | 23        |
| 5. Chuỗi ràng buộc phiên bản                                   | 24        |
| 6. Các vấn đề khác (tóm tắt)                                   | 25        |
| <b>VI. Phân tích hiệu năng và mục tiêu độ trễ dưới 2 giây</b>  | <b>26</b> |
| 1. Định vị kiến trúc - mục tiêu đến từ đâu                     | 26        |
| 2. Kết quả đo và mô hình chi phí                               | 27        |
| 3. Đây không phải lỗi của mã nguồn                             | 28        |
| 4. Giải pháp chính thức không áp dụng được                     | 29        |
| 5. Đây có phải vấn đề về công nghệ không?                      | 29        |
| 6. Đây có phải vấn đề về logic không?                          | 30        |
| 7. Hướng đi chưa được xem xét: phát hiện tại biên              | 30        |
| 8. Các phương án và đánh giá                                   | 31        |
| 9. Kết luận chương                                             | 31        |
| <b>VII. Kết quả đạt được</b>                                   | <b>31</b> |
| 1. Độ chính xác phát hiện                                      | 31        |
| 2. Kết quả theo từng loại tấn công                             | 32        |
| 3. Tính tái lập của kết quả                                    | 33        |
| 4. Pipeline không làm sai lệch dữ liệu                         | 33        |
| 5. Hiệu quả lưu trữ                                            | 34        |
| 6. Hiệu năng                                                   | 34        |
| 7. Dashboard                                                   | 35        |
| <b>VIII. Kết luận</b>                                          | <b>36</b> |
| 1. Những gì đã đạt được                                        | 36        |
| 2. Những hạn chế                                               | 37        |
| 3. Hướng phát triển                                            | 37        |
| 4. Nhận định cuối                                              | 38        |
| <b>TÀI LIỆU THAM KHẢO</b>                                      | <b>39</b> |

# I. Giới thiệu

## 1. Bối cảnh

Các hệ thống công nghệ thông tin hiện đại sinh ra khối lượng dữ liệu mạng rất lớn và liên tục. Trong bối cảnh đó, một cuộc tấn công không còn là sự kiện đơn lẻ dễ nhận biết bằng mắt thường, mà là một tín hiệu nhỏ ẩn trong hàng trăm nghìn bản ghi lưu lượng bình thường mỗi phút. Việc phát hiện hành vi bất thường càng sớm thì thiệt hại càng được giảm thiểu - nhưng “sớm” ở đây phải được định nghĩa và đo đạc một cách nghiêm túc, chứ không phải khẳng định cảm tính.

Đây chính là điểm giao nhau giữa hai lĩnh vực: an toàn thông tin cung cấp bài toán và tiêu chí đánh giá, còn kỹ thuật dữ liệu (data engineering) cung cấp hạ tầng để xử lý dòng dữ liệu đủ nhanh và độ tin cậy đủ cao.

## 2. Vấn đề cần giải quyết

Đề án này hướng tới giải quyết bài toán sau:

Làm thế nào để xây dựng một đường ống (pipeline) dữ liệu thời gian thực có khả năng tiếp nhận lưu lượng mạng liên tục, đánh giá từng bản ghi theo một tập luật phát hiện, lưu trữ kết quả một cách tiết kiệm, và cung cấp số liệu đánh giá độ chính xác có thể kiểm chứng được?

Bài toán này bao gồm 4 thách thức con:

- **Thông lượng:** Hệ thống phải tiếp nhận và xử lý dữ liệu liên tục mà không tích lũy độ trễ vô hạn. Một pipeline xử lý chậm hơn tốc độ dữ liệu đến sẽ tụt hậu ngày càng xa, và cảnh báo trở nên vô nghĩa.
- **Độ trễ:** Khoảng thời gian từ khi một bản ghi được sinh ra đến khi nó được đánh giá và lưu trữ phải đủ nhỏ để cảnh báo còn giá trị hành động.
- **Độ chính xác đo được:** Một IDS không chỉ cần bắt được tấn công (recall) mà còn không được tạo báo động giả quá nhiều (precision). Quan trọng hơn, các chỉ số này phải tính toán từ dữ liệu đã lưu, chứ không phải ước lượng.
- **Chi phí lưu trữ:** Lưu trữ toàn bộ lưu lượng mạng là bất khả thi về lâu dài. Hệ thống cần một thiết kế lưu trữ chọn lọc mà vẫn giữ đủ thông tin để tính đầy đủ các chỉ số đánh giá.

## 3. Mục tiêu và phạm vi

Mục tiêu:

- Xây dựng pipeline hoàn chỉnh: sinh dữ liệu => truyền tải => xử lý => lưu trữ => trực quan hóa
- Phát hiện sáu nhóm hành vi tấn công bằng phương pháp dựa trên luật (rule-based)

- Đo đạc và báo cáo độ chính xác, thông lượng, độ trễ bằng số liệu thực tế
- Trực quan hóa hoạt động hệ thống theo thời gian thực

Phạm vi:

- Dữ liệu mạng được mô phỏng, không phải lưu lượng thật
- Phương pháp phát hiện là rule-based, không sử dụng ML
- Hệ thống chạy trên một máy tính cá nhân, không phải cụm phân tán

Sáu nhóm hành vi tấn công được phát hiện:

| Nhóm tấn công             | Đặc trưng nhận dạng                                                      |
|---------------------------|--------------------------------------------------------------------------|
| DDoS                      | Số kết nối rất cao, tỷ lệ lỗi SYN cao, payload gần bằng không            |
| Port Scanning             | Quét nhiều dịch vụ khác nhau, tỷ lệ cùng-dịch-vụ thấp, tỷ lệ từ chối cao |
| Brute Force               | Nhiều lần đăng nhập thất bại liên tiếp trên cùng một dịch vụ             |
| Malware Phoning Home (C2) | Kết nối nhỏ, định kỳ, tập trung vào một host/dịch vụ                     |
| Data Exfiltration         | Lưu lượng đi ra rất lớn, phản hồi vào rất nhỏ, đã đăng nhập              |
| Malicious File Download   | Lưu lượng vào rất lớn với yêu cầu ra rất nhỏ                             |

*Bảng I-1: Các nhóm tấn công và đặc điểm nhận dạng*

#### 4. Những thay đổi so với đề cương

Trong quá trình thực hiện, một số quyết định công nghệ đã thay đổi so với đề cương ban đầu. Các thay đổi này đều xuất phát từ lý do kỹ thuật cụ thể và được phân tích chi tiết tại chương III.

**Lưu ý quan trọng về thay đổi lược đồ dữ liệu:** Đề cương dự kiến sinh dữ liệu có Source IP, Destination IP, Session ID, User ID. Bộ dữ liệu NSL-KDD không chứa bất kỳ trường định danh nào - đây là thiết kế có chủ đích của bộ dữ liệu, nhằm bảo vệ quyền riêng tư. Hệ quả là hệ thống không thể liên kết các sự kiện theo kẻ tấn công, chỉ có thể liên kết theo cửa sổ thời gian và chữ ký tấn công. Đây là một giới hạn có thật, được ghi nhận tại chương VIII.

| Thành phần      | Đề cương                          | Triển khai thực tế     | Lý do tóm tắt                                                                    |
|-----------------|-----------------------------------|------------------------|----------------------------------------------------------------------------------|
| Lưu trữ         | PostgreSQL                        | ClickHouse             | Cơ sở dữ liệu cột, tối ưu cho nhật ký sự kiện chi ghi thêm                       |
| Dashboard       | Streamlit                         | Apache Superset        | Có sẵn kết nối ClickHouse, không cần viết mã giao diện                           |
| Lược đồ dữ liệu | Tự định nghĩa (IP, port, session) | NSL-KDD (41 đặc trưng) | Bộ dữ liệu chuẩn, có nhãn sẵn, phục vụ đánh giá khách quan                       |
| Hạ tầng         | 2 máy ảo Azure                    | Docker Compose cục bộ  | Đơn giản hóa, loại bỏ cấu hình mạng/SSH/TLS                                      |
| Mục tiêu độ trễ | < 2 giây                          | ~ 3.3 giây (đo được)   | Phân tích cho thấy mục tiêu không khả thi với kiến trúc hiện tại - xem chương VI |

Bảng I-2: So sánh các thay đổi giữa đề cương và triển khai thực tế

## II. Cơ sở lý thuyết

### 1. Hệ thống phát hiện xâm nhập

**Hệ thống phát hiện xâm nhập (Intrusion Detection System - IDS)** là hệ thống giám sát lưu lượng mạng hoặc hệ thống nhằm phát hiện các hành vi vi phạm chính sách bảo mật hoặc dấu hiệu tấn công. Khác với tường lửa (firewall) hoạt động theo nguyên tắc cho phép/chặn dựa trên quy tắc tĩnh, IDS phân tích nội dung và ngữ cảnh của lưu lượng để nhận diện hành vi bất thường.

#### a. Phân loại theo vị trí triển khai

**NIDS (Network-based IDS)** đặt tại các điểm trọng yếu của mạng, giám sát lưu lượng đi qua. Ưu điểm là bao quát nhiều máy cùng lúc; nhược điểm là không thấy được hoạt động nội bộ của từng máy và gặp khó khăn với lưu lượng đã mã hóa.

**HIDS (Host-based IDS)** cài đặt trên từng máy chủ, giám sát nhật ký hệ thống, tính toán vận tệp tin, tiến trình đang chạy. Ưu điểm là thấy được chi tiết; nhược điểm là chi phí triển khai và quản lý trên quy mô lớn.

Hệ thống trong đồ án này thuộc nhóm NIDS, phân tích các bản ghi tóm tắt kết nối mạng.

b. Phân loại theo phương pháp phát hiện

**Signature-based Detection (phát hiện dựa trên chữ ký)** so khớp lưu lượng với một tập mẫu tấn công đã biết.

- Ưu điểm: Độ chính xác cao với các mẫu tấn công đã biết, tỷ lệ báo động giả thấp, kết quả giải thích được - luôn chỉ ra được luật nào đã kích hoạt
- Nhược điểm: Không phát hiện ra được các mẫu tấn công mới (zero-day), cần cập nhật tập luật thường xuyên

**Anomaly-based Detection (phát hiện dựa trên bất thường)** xây dựng mô hình hành vi “bình thường” và cảnh báo khi có sai lệch đáng kể.

- Ưu điểm: Có khả năng phát hiện các mẫu tấn công chưa từng được ghi nhận
- Nhược điểm: Tỷ lệ báo động giả cao, cần giai đoạn huấn luyện, kết quả khó giải thích

Đồ án lựa chọn hướng rule-based, một dạng của signature-based detection. Lý do lựa chọn được trình bày tại chương II mục 3.

## 2. Phát hiện dựa trên luật

Trong phương pháp rule-based, mỗi luật là một tập điều kiện trên các đặc trưng của bản ghi. Một luật có cấu trúc:

- rule\_id - định danh duy nhất của luật
- attack\_type - loại tấn công mà luật này nhận diện
- severity - mức độ nghiêm trọng (low/medium/high/critical)
- conditions[] - danh sách điều kiện, được kết hợp bằng phép AND

Ví dụ luật phát hiện DDoS được sử dụng trong hệ thống:

```
{
  "rule_id": "ddos_flood",
  "attack_type": "ddos",
  "severity": "critical",
  "conditions": [
    {"field": "count", "op": ">=", "value": 150},
    {"field": "serror_rate", "op": ">=", "value": 0.7},
    {"field": "dst_bytes", "op": "<=", "value": 20}
  ]
}
```

Hình II-1: Ví dụ luật phát hiện DDoS



Luật này diễn giải thành: “Một kết nối bị coi là DDoS nếu có từ 150 trở lên cùng hoạt động trong cửa sổ thời gian, tỷ lệ lỗi SYN từ 70% trở lên, và dữ liệu phản hồi không quá 20 byte.” Ba điều kiện trên mô tả đặc trưng của một cuộc tấn công DDoS.

Các điều kiện trong một luật được kết hợp bằng AND; các luật khác nhau được kết hợp bằng OR. Nhiều luật có thể cùng trỏ tới một `attack_type` - khi đó chỉ cần một chữ ký bất kỳ khớp là đủ để xếp bản ghi vào loại tấn công đó. Đây cũng là lý do `matched_attack_types` được lưu dưới dạng mảng chứ không phải một giá trị đơn — một bản ghi có thể khớp nhiều luật thuộc nhiều loại tấn công khác nhau cùng lúc.

Trong tập dữ liệu có nhãn, mỗi bản ghi mang trường `label` cho biết đó là lưu lượng bình thường hay loại tấn công nào. Trường này tuyệt đối không bao giờ được sử dụng trong quá trình phát hiện. Một IDS thực tế không bao giờ biết trước sự thật; nếu luật đọc nhãn, độ chính xác sẽ đạt 100% một cách vô nghĩa.

Trong hệ thống, nguyên tắc này được ép buộc bởi mã nguồn: trường `label` nằm trong danh sách trường bị cấm, và bất kỳ luật nào tham chiếu tới nó sẽ khiến hệ thống báo lỗi ngay khi khởi động. Nhãn chỉ được dùng sau khi phát hiện xong để chấm điểm.

### 3. Lý do chọn phương pháp rule-based

| Tiêu chí                   | Rule-based                        | Machine Learning                         |
|----------------------------|-----------------------------------|------------------------------------------|
| Khả năng giải thích        | Cao - chỉ rõ luật nào kích hoạt   | Thấp - khó truy vết quyết định           |
| Thời gian phát triển       | Ngắn                              | Dài (thu thập, huấn luyện, tinh chỉnh)   |
| Chi phí tính toán khi chạy | Rất thấp                          | Cao hơn                                  |
| Phát hiện tấn công mới     | Không                             | Có tiềm năng                             |
| Phù hợp trọng tâm đồ án    | Có - trọng tâm là hạ tầng dữ liệu | Không - sẽ chuyển trọng tâm sang mô hình |

*Bảng II-1: So sánh Rule-based vs Machine Learning*

Trọng tâm của đồ án là kỹ thuật dữ liệu: thiết kế pipeline, đảm bảo thông lượng, quản lý lược đồ, xử lý lỗi, đo đặc hiệu năng. Rule-based cho phép tập trung vào các vấn đề đó, đồng thời giữ được tính giải thích được - vốn là yêu cầu quan trọng trong an toàn thông tin, nơi mỗi cảnh báo cần được phân tích viên xác minh.

### 4. Các chỉ số đánh giá

Kết quả phát hiện của mỗi bản ghi rơi vào một trong bốn nhóm:

|                      | Dự đoán: Tấn công                  | Dự đoán: Bình thường         |
|----------------------|------------------------------------|------------------------------|
| Thực tế: Tấn công    | TP- True Positive                  | FN - False Negative (bỏ sót) |
| Thực tế: Bình thường | FP - False Positive (báo động giả) | TN (True Negative)           |

Bảng II-2: Ma trận nhầm lẫn (Confusion Matrix)

Từ bốn giá trị trên, các chỉ số được tính:

- **Recall (độ nhạy):** Trong số các cuộc tấn công thật sự, hệ thống phát hiện được bao nhiêu phần trăm:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

- **Precision (độ chính xác của cảnh báo):** Trong số các cảnh báo phát ra, bao nhiêu phần trăm là đúng

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- **F1-score:** Trung bình điều hòa (harmonic mean) của hai chỉ số trên

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

- **Tỷ lệ báo động giả (False Positive Rate):** Trong toàn bộ lưu lượng bình thường, bao nhiêu phần trăm bị gắn cờ nhầm:

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

Lưu ý là chỉ số Accuracy không được sử dụng trong phạm vi đề án này. Chỉ số Accuracy - tính bằng  $(\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$  - thường được sử dụng nhưng không phù hợp với bài toán IDS do hiện tượng mất cân bằng lớp (class imbalance).

Trong hệ thống này, lưu lượng bình thường chiếm 95%, do đó một hệ thống không phát hiện gì cả - luôn trả lời “bình thường” - vẫn đạt Accuracy = 95%.

Hệ thống thực tế đạt Accuracy = 99.4%, tức chỉ hơn ngưỡng cơ sở này 4.4%. Con số 99.4% nghe rất ấn tượng nhưng che giấu thực tế đó.

Ngược lại, Recall và FPR không thể bị “đánh lừa” bằng cách không làm gì: một hệ thống im lặng sẽ có Recall = 0%. Vì vậy hai chỉ số này là căn cứ đánh giá chính trong báo cáo, còn Accuracy chỉ được nêu kèm ngưỡng cơ sở để tránh gây hiểu lầm.

## 5. Bộ dữ liệu NSL-KDD

Hệ thống sử dụng lược đồ NSL-KDD - phiên bản cải tiến của bộ dữ liệu KDD Cup 1999, đã loại bỏ các bản ghi trùng lặp gây sai lệch đánh giá. Đây là bộ dữ liệu chuẩn được sử dụng rộng rãi trong nghiên cứu IDS.

Mỗi bản ghi gồm 41 đặc trưng, chia thành bốn nhóm:

| Nhóm                                    | Số lượng | Ví dụ                                                  | Ý nghĩa                     |
|-----------------------------------------|----------|--------------------------------------------------------|-----------------------------|
| Đặc trưng cơ bản                        | 9        | duration, protocol_type, service, src_bytes, dst_bytes | Thuộc tính bản thân kết nối |
| Đặc trưng nội dung                      | 13       | num_failed_login, logged_in, root_shell                | Dấu hiệu trong phần dữ liệu |
| Đặc trưng của lưu lượng (cửa sổ 2 giây) | 9        | count, srv_count, serror_rate, diff_srv_rate           | Thống kê theo thời gian     |
| Đặc trưng theo host đích                | 10       | dst_host_count, dst_host_same_srv_rate                 | Thống kê tích lũy theo host |

*Bảng II-3: Các nhóm đặc trưng của lược đồ NSL-KDD*

Một điểm quan trọng cần hiểu rõ: mỗi bản ghi NSL-KDD đã là một bản tóm tắt kết nối kèm ngữ cảnh cửa sổ, không phải một gói tin đơn lẻ. Ví dụ trường count mang ý nghĩa “số kết nối tới cùng host trong 2 giây vừa qua”. Điều này giải thích vì sao các luật ngưỡng có thể hoạt động trên từng bản ghi độc lập.

Hệ thống bổ sung hai trường ngoài 41 đặc trưng gốc: timestamp (thời điểm sinh, dùng để đo độ trễ) và label (nhãn thực tế, chỉ dùng để chấm điểm).

## 6. Kiến trúc phân tầng của một hệ thống IDS thực tế

Đây là phần lý thuyết quan trọng nhất để hiểu kết quả tại chương VI.

Trong các hệ thống IDS triển khai thực tế, việc phát hiện và phân tích nằm ở hai tầng khác nhau, với yêu cầu độ trễ khác nhau hàng nghìn lần:

- Tầng cảm biến (Sensor)
  - Snort, Suricata, Zeek - chạy trực tiếp trên đường truyền
  - Nhiệm vụ: Quyết định một gói tin/kết nối có độc hại không
  - Độ trễ: 350 micro-giây đến vài mili-giây
  - Đặc điểm: Không trạng thái, xét từng bản ghi độc lập
- Tầng phân tích (Analytics/SIEM)

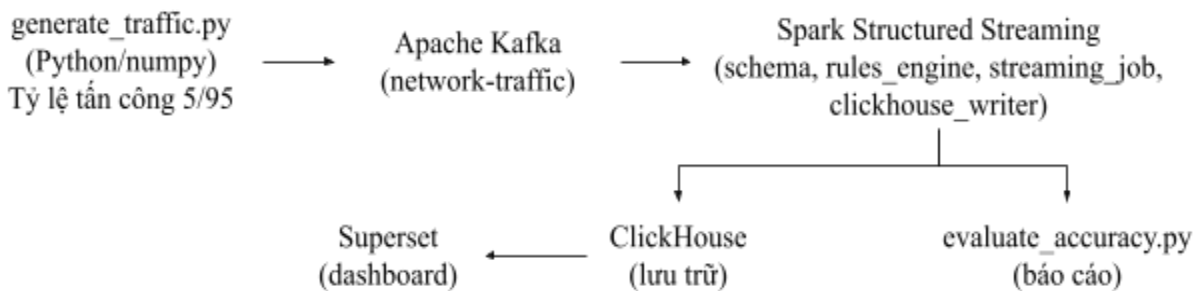
- Kafka + Spark/Flink + kho dữ liệu + dashboard
- Nhiệm vụ: Liên kết, làm giàu, thống kê, cảnh báo tổng hợp
- Độ trễ: Vài giây đến vài phút
- Đặc điểm: Cần ngữ cảnh thời gian và kết nối chéo nguồn

Sự phân chia này không phải tùy chọn thiết kế mà xuất phát từ bản chất công việc: quyết định một bản ghi có độc hại hay không là công việc không trạng thái, trên từng bản ghi, có thể thực hiện tại chỗ trong mili-giây. Ngược lại, việc phát hiện “cùng một địa chỉ đã quét 50 cổng trong 10 phút vừa qua và vừa tải về một tệp lạ” đòi hỏi ngữ cảnh thời gian và dữ liệu từ nhiều nguồn - điều mà một cảm biến đơn lẻ không thể làm.

Hệ thống trong đồ án này, về mặt kiến trúc, thuộc tầng thứ hai. Nhận định này sẽ được sử dụng để phân tích mục tiêu về độ trễ tại Chương VI.

### III. Lựa chọn công nghệ

#### 1. Tổng quan kiến trúc



Hình III-1: Tổng quan kiến trúc của đồ án

#### 2. Apache Kafka - Tầng tiếp nhận dữ liệu

**Vai trò:** Hàng đợi phân tán, tiếp nhận dữ liệu từ bộ sinh và cung cấp cho Spark.

**Lý do lựa chọn:**

- **Tách rời producer và consumer:** Bộ sinh dữ liệu và Spark hoạt động độc lập. Nếu Spark dừng để bảo trì, dữ liệu vẫn được Kafka lưu giữ, không mất mát. Đây là tính chất mà một kết nối trực tiếp không có.
- **Khả năng phát lại (replay):** Kafka lưu trữ dữ liệu theo offset. Nhờ đó có thể xử lý lại toàn bộ luồng dữ liệu từ đầu - tính năng đã được sử dụng nhiều lần trong quá trình kiểm thử để đánh giá lại độ chính xác trên cùng một tập dữ liệu.
- **Đảm bảo thứ tự và độ bền:** Trong một phần vùng, thứ tự bản ghi được giữ nguyên; dữ liệu được ghi xuống đĩa trước khi xác nhận.
- **Hỗ trợ sẵn từ Spark:** Spark Structured Streaming có connector Kafka chính thức với cơ chế quản lý offset tự động.

**Cấu hình sử dụng:** Kafka chế độ KRaft (single-node), không cần Zookeeper - giảm một thành phần cần phải quản lý.

### 3. Apache Spark Structured Streaming - Tầng xử lý

**Vai trò:** Đọc dữ liệu từ Kafka, phân tích cú pháp, đánh giá luật, ghi kết quả.

**Lý do lựa chọn:**

- **Mô hình lập trình thống nhất:** Cùng một API DataFrame được dùng cho cả xử lý theo lô và theo luồng. Nhờ đó, `rules_engine.py` được kiểm thử ngoại tuyến bằng dữ liệu tĩnh rồi đưa thẳng vào pipeline luồng mà không cần viết lại.
- **Đảm bảo exactly-once:** Cơ chế checkpoint ghi lại offset Kafka đã xử lý. Sau sự cố, công việc tiếp tục đúng vị trí, không bỏ sót cũng không xử lý lại.
- **Tối ưu hóa bằng Catalyst:** Các biểu thức được biên dịch thành mã Java qua whole-stage codegen (WSCG). Ưu điểm này chỉ phát huy nếu không dùng Python UDF - nguyên tắc thiết kế được tuân thủ nghiêm ngặt trong `rules_engine.py` (xem chương IV mục 4).

**Hạn chế đã biết:** Spark xử lý theo micro-batch, tức gom dữ liệu lại thành từng lô nhỏ theo chu kỳ. Đây là nguồn gốc của giới hạn độ trễ được phân tích tại chương VI.

### 4. ClickHouse - Tầng lưu trữ

**Bản chất dữ liệu cần lưu:** một nhật ký sự kiện chỉ ghi thêm (append-only), không bao giờ cập nhật hay xóa bản ghi cũ. Các truy vấn chủ yếu là tổng hợp trên toàn bộ bảng: đếm theo nhãn, tính tỷ lệ theo loại tấn công, thống kê phân vị độ trễ.

| Tiêu chí                                            | PostgreSQL                           | ClickHouse                                     |
|-----------------------------------------------------|--------------------------------------|------------------------------------------------|
| Mô hình lưu trữ                                     | Theo dòng (row-oriented)             | Theo cột (column-oriented)                     |
| Tối ưu cho                                          | Giao dịch OLTP, đọc/ghi từng bản ghi | Tổng hợp phân tích OLAP                        |
| Nén dữ liệu                                         | Trung bình                           | Cao (dữ liệu cùng cột đồng nhất)               |
| Truy vấn COUNT BY/<br>GROUP BY trên hàng triệu dòng | Chậm hơn                             | Rất nhanh                                      |
| Ghi theo lô lớn                                     | Tốt                                  | Rất tốt (MergeTree được thiết kế cho việc này) |

*Bảng III-1: So sánh PostgreSQL với ClickHouse*

**Lý do quyết định thay đổi:**

Một truy vấn điển hình của hệ thống chỉ đọc 2-3 cột trên hàng trăm nghìn dòng (ví dụ label, is\_detection). Cơ sở dữ liệu theo dòng buộc phải đọc toàn bộ hơn 40 cột của mỗi dòng; cơ sở dữ liệu theo cột chỉ đọc đúng các cột cần thiết. Với dữ liệu nhật ký bảo mật, chênh lệch này rất đáng kể.

Ngoài ra, ClickHouse triển khai đơn giản hơn nhiều so với các giải pháp OLAP khác (như StarRocks vốn cần cấu hình FE/BE riêng biệt) - chỉ một container duy nhất, phù hợp với đồ án cá nhân.

Engine MergeTree được sử dụng với khóa sắp xếp ORDER BY (label, timestamp), tối ưu cho các truy vấn lọc theo nhãn.

**5. Apache Superset - Tầng trực quan hóa**

| Tiêu chí                      | Streamlit                       | Apache Superset               |
|-------------------------------|---------------------------------|-------------------------------|
| Cách xây dựng biểu đồ         | Viết mã Python cho từng biểu đồ | Cấu hình qua giao diện        |
| Kết nối ClickHouse            | Phải tự viết                    | Có sẵn qua clickhouse-connect |
| Tự động làm mới               | Phải tự triển khai              | Có sẵn (thiết lập chu kỳ)     |
| Bộ lọc, lọc chéo giữa biểu đồ | Phải tự viết                    | Có sẵn                        |

*Bảng III-2: So sánh Streamlit và Apache Superset*

**Lý do quyết định thay đổi:** Nhu cầu của đồ án là một dashboard giám sát - bảng cảnh báo, các ô chỉ số, biểu đồ theo thời gian, tự động làm mới. Đây chính xác là bài toán mà Superset được thiết kế để giải. Chọn Streamlit đồng nghĩa với việc viết lại bằng mã Python những chức năng Superset đã có sẵn, mà không mang lại giá trị nào cho trọng tâm đồ án.

Superset cũng bổ sung các tính năng khó tự xây dựng: lọc chéo giữa các biểu đồ, phân trang bảng, định dạng bảng có điều kiện và quản lý bộ dữ liệu ảo (virtual dataset).

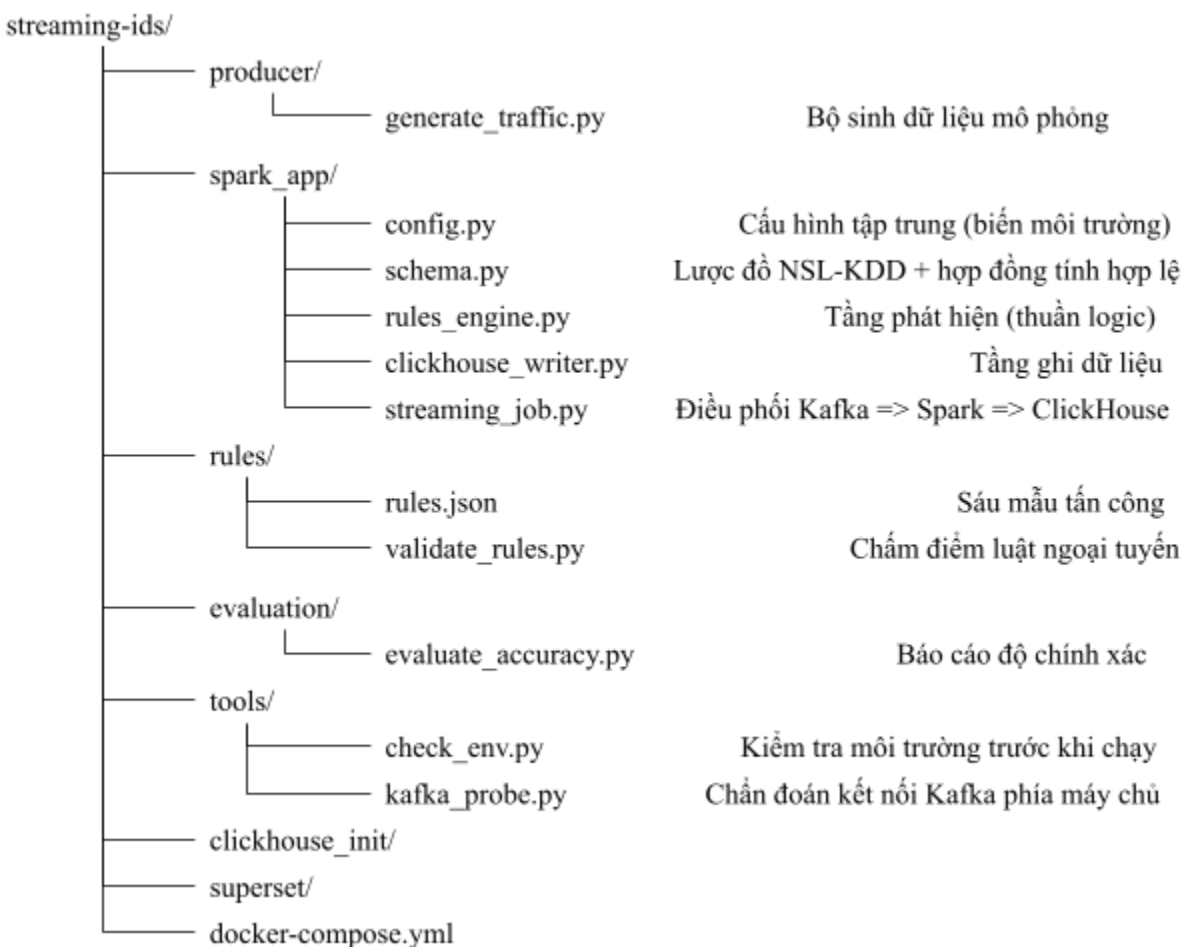
**6. Docker Compose - Tầng hạ tầng**

Ba dịch vụ trạng thái (Kafka, Clickhouse, Superset) chạy trong container; Spark và bộ sinh dữ liệu chạy trực tiếp trên máy chủ.

**Lý do phân chia như vậy:**

- **Container hóa các dịch vụ trạng thái:** Cài đặt và cấu hình Kafka, ClickHouse, Superset thủ công tốn nhiều thời gian và khó tái lập. Docker cho phép dựng lại toàn bộ hạ tầng bằng một lệnh.
- **Chạy Spark trực tiếp:** Spark ở chế độ local là một thư viện, không phải dịch vụ. Đóng gói nó vào container sẽ khiến mỗi lần sửa mã phải build lại image, làm chậm quá trình phát triển đáng kể.

**Đánh đổi đã ghi nhận:** Spark chạy trực tiếp nghĩa là nó sẽ kế thừa môi trường của máy chủ - phiên bản Python, phiên bản Java, hệ điều hành. Nhiều vấn đề ở chương V là hệ quả trực tiếp của lựa chọn này.

**IV. Xây dựng hệ thống****1. Cấu trúc mã nguồn**

Hình IV-1: Cấu trúc thư mục mã nguồn của đồ án

## 2. Bộ sinh dữ liệu

**Nguyên tắc thiết kế:** Mỗi lớp lưu lượng có phân phối đặc trưng riêng, được xây dựng theo hành vi thực tế của loại lưu lượng đó, chứ không phải giá trị ngẫu nhiên đồng đều.

Ví dụ, lưu lượng DDoS được sinh ra với duration gần bằng không, error\_rate rất cao và dst\_bytes gần bằng không - phản ánh đặc trưng của một đợt tấn công DDoS. Lưu lượng data exfiltration có src\_bytes rất lớn nhưng dst\_bytes nhỏ.

**Tối ưu hiệu năng:** Toàn bộ việc sinh dữ liệu được vector hóa bằng numpy - sinh cả một mảng 5,000 bản ghi cùng lúc thay vì lặp từng bản ghi. Kết quả đo được với cấu hình máy tính cá nhân (i5-13420H 8 nhân/12 luồng) với 8 workers là ~169,000 bản ghi/giây.

Kích thước bản ghi đo được: ~1,120 byte (JSON đã tuần tự hóa).

**Điều chỉnh mục tiêu thông lượng:** Mục tiêu ban đầu 1.000.000 bản ghi/giây cần khoảng 55 nhân CPU, mạng 10GbE và cụm Kafka nhiều broker - không khả thi trên phần cứng cá nhân. Tốc độ demo được đặt ở mức 1 GB/phút (~14.880 bản ghi/giây) để không gây quá tải máy trong quá trình phát triển.

## 3. Hợp đồng lược đồ dữ liệu

schema.py định nghĩa NSL\_KDD\_SCHEMA - một StructType của Spark phản ánh chính xác cấu trúc JSON của bộ sinh: 41 đặc trưng + timestamp + label.

**Một quyết định kiểu dữ liệu đáng chú ý:** Trường duration được khai báo là DoubleType chứ không phải IntegerType, vì bộ sinh xuất ra số thực đã làm tròn (ví dụ 12.34). Nếu khai báo là số nguyên, hàm from\_json của Spark sẽ âm thầm trả về null cho toàn bộ bản ghi mà không báo lỗi nào. Lỗi này rất khó phát hiện nếu không kiểm tra kỹ.

Lược đồ này còn là nguồn duy nhất sinh ra câu lệnh DDL tạo bảng ClickHouse. Nhờ đó cấu trúc bảng không bao giờ lệch khỏi lược đồ dữ liệu: thêm một trường trong schema.py thì cột tương ứng tự động xuất hiện trong bảng.

## 4. Bộ máy luật - thiết kế không dùng UDF

rules\_engine.py là bộ phận duy nhất chịu trách nhiệm cho việc quyết định một bản ghi có phải tấn công hay không. Module này không biết gì về Kafka, Spark, checkpoint hay ClickHouse.

Ranh giới này cho phép chấm điểm tập luật hoàn toàn ngoại tuyến - validate\_rules.py nạp trực tiếp rules\_engine, đưa dữ liệu từ bộ sinh vào, và báo cáo tỷ lệ recall cùng tỷ lệ báo động giả trong khoảng 30 giây, không cần Kafka hay ClickHouse chạy. Đây là vòng phản hồi nhanh dùng sau mỗi lần sửa rules.json.



## Nguyên tắc quan trọng nhất: Không dùng Python UDF

Toàn bộ luật được biên dịch thành biểu thức Column gốc của Spark (when, array, filter, array\_max). Lý do chính không nằm ở tốc độ mà ở khả năng tối ưu hóa: một UDF Python là hộp đen đối với Catalyst - bộ tối ưu hóa không thể nhìn vào bên trong để sắp xếp lại hay hợp nhất vào WSCG, nên mọi tối ưu hóa dừng lại ở ranh giới UDF.

Trên lý thuyết, mỗi dòng đi qua UDF phải vượt ranh giới tiến trình JVM - Python kèm hai lần tuần tự hóa. Chi phí trên mỗi dòng là cố định và nhỏ, nhưng nó nhân với số dòng mỗi giây - nên nó trở thành nút thắt khi cần xử lý thông lượng cao. Lấy ví dụ, ở tốc độ demo ~15,000 bản ghi/giây, phần chi phí này còn khiêm tốn; nó chỉ trở nên quyết định khi hướng tới mức hàng trăm nghìn bản ghi/giây. (Trong phạm vi đề án không đo phương án dùng UDF để đối chứng và lấy số liệu thực.)

Nguyên tắc này được kiểm chứng bằng khẳng định tự động: bộ kiểm thử của module xác nhận kế hoạch thực thi không chứa nút BatchEvalPython hay ArrowEvalPython - hai thứ Catalyst bắt buộc phải chèn vào khi gặp UDF Python. Nếu có ai vô tình thêm một UDF, bài kiểm thử sẽ báo lỗi ngay.

**Xử lý giá trị null:** Một bản ghi lỗi có thể lọt qua bước phân tích cú pháp với các trường bằng null. Trong SQL, phép so sánh `NULL >= 150` không trả về False mà trả về NULL. Nếu không xử lý, `is_detection` sẽ nhận giá trị NULL thay vì False, và cột `is_detection` Bool trong ClickHouse không cho phép NULL. Vì vậy mọi điều kiện được bọc trong `coalesce(..., False)`.

**Kiểm định tập luật nghiêm ngặt:** Hệ thống từ chối khởi động nếu `rules.json` có: tên trường không tồn tại, toán tử không hỗ trợ, `rule_id` trùng lặp, mức độ nghiêm trọng không hợp lệ, danh sách điều kiện rộng, kiểu dữ liệu không khớp, hoặc bất kỳ tham chiếu nào tới trường label. Triết lý ở đây là: Một luật sai chính tả tên trường không bao giờ khớp - và một luật không bao giờ khớp là lỗi im lặng nguy hiểm và khó phát hiện hơn nhiều so với một thông báo lỗi khi khởi động.

## 5. Thiết kế lưu trữ hai bảng

Đây là một trong những quyết định thiết kế đáng chú ý nhất của hệ thống.

### a. Bảng detection

Lưu đầy đủ 43 cột đặc trưng cùng kết quả phát hiện, nhưng chỉ với các bản ghi đáng lưu: `label != 'normal'` HOẶC `is_detection = True`, tức mọi tấn công thật và mọi bản ghi bị gán cờ. Lưu lượng bình thường được bỏ qua và không được lưu trữ.

**Kết quả đo:** bảng này chỉ chiếm ~5.56% tổng dữ liệu đi vào.

## b. Bảng traffic\_counts

Mỗi micro-batch ghi một vài dòng rất nhỏ: (batch\_id, batch\_time, label, record\_count).

Vì sao cần hai bảng này? Đây là điểm mấu chốt:

- Bảng detections có thể tính được Recall và Precision - vì mọi tấn công và mọi cảnh báo đều nằm trong đó. Nhưng nó không thể tính được FPR, vì tỷ lệ này cần biết tổng số bản ghi thường đã đi qua hệ thống - mà lưu lượng bình thường được bỏ qua lại không được lưu ở đâu cả.
- Bảng traffic\_counts cung cấp chính xác mẫu số đó với chi phí lưu trữ gần như bằng không: chỉ lưu số đếm, không lưu bản ghi.

Đây là ví dụ điển hình của việc thiết kế lưu trữ phải xuất phát từ câu hỏi cần trả lời, chứ không phải từ dữ liệu sẵn có.

## 6. Xử lý bản ghi lỗi - phân loại hai nhóm

Bản ghi lỗi được phân ra thành hai nhóm riêng biệt, không phải một:

| Nhóm                      | Ý nghĩa                                             | Chỉ ra vấn đề ở đâu                                                   |
|---------------------------|-----------------------------------------------------|-----------------------------------------------------------------------|
| __malformed_unparseable__ | Không phân tích được JSON - không còn gì nguyên vẹn | Truyền tải: đường truyền bị cắt, lỗi mã hóa, producer chết giữa chừng |
| __malformed_incomplete__  | JSON hợp lệ nhưng thiếu trường hoặc sai kiểu        | Sinh dữ liệu/lịch lược đồ: bản tin nguyên vẹn nhưng sai từ khi tạo    |

*Bảng IV-1: Phân biệt hai nhóm dữ liệu lỗi*

**Vì sao phải tách hai nhóm:** Lỗi truyền tải và lỗi lược đồ trông giống hệt nhau trong một bộ đếm duy nhất, nhưng cách khắc phục hoàn toàn khác nhau. Nhóm nào tăng lên sẽ cho biết cần mở tệp nào để sửa.

Số đếm của cả hai nhóm được ghi vào chính bảng traffic\_counts, nên tỷ lệ lỗi có thể vẽ biểu đồ ngay cạnh lưu lượng thật mà không cần thêm bảng nào.

## 7. Đo độ trễ

Bảng detections mang hai cột processed\_at và latency\_seconds, cho phép báo cáo các số đo thực tế thay vì khẳng định suông.

latency\_seconds được tính bằng hiệu giữa thời điểm ghi vào ClickHouse và trường timestamp của chính bản ghi - tức thời điểm bộ sinh tạo ra nó. Nhờ vậy con số phản ánh độ trễ đầu-cuối thật sự, không chỉ thời gian xử lý nội bộ của Spark.

## V. Các vấn đề gặp phải và giải pháp

Chương này trình bày các vấn đề kỹ thuật đã gặp trong quá trình xây dựng. Năm vấn đề đầu được phân tích chi tiết vì chúng mang giá trị kỹ thuật hoặc phương pháp luận; các vấn đề còn lại được liệt kê ngắn gọn.

### 1. Truy vấn luồng bị treo do Catalyst constraint propagation

Đây là lỗi khó phát hiện nhất và có giá trị kỹ thuật cao nhất toàn bộ đề án.

#### Hiện tượng

Sau khi toàn bộ hạ tầng đã hoạt động, công việc streaming khởi động sạch sẽ: nạp đủ 6 luật, tạo bảng ClickHouse thành công, báo Streaming IDS started. Trạng thái truy vấn vẫn báo Processing new data. Truy vấn vẫn trả isActive.

Nhưng không một micro-batch nào hoàn thành. Không lỗi, không cảnh báo, không dữ liệu. Trong hơn 80 giây.

#### Quá trình chẩn đoán

Nghi vấn đầu tiên là Kafka. Để loại trừ, một công cụ chẩn đoán riêng (tools/kafka\_probe.py) được viết để kiểm tra khả năng tiêu thụ dữ liệu từ phía máy chủ - điều mà chưa phép thử nào trước đó kiểm tra (nội dung topic được kiểm tra từ bên trong container, còn producer chỉ chứng minh khả năng ghi).

Kết quả: Kafka đọc được 1,372,349 bản tin trong 10 giây => Kafka hoàn toàn khỏe mạnh.

Bước tiếp theo là lấy thread dump của driver từ giao diện Spark UI. Luồng thực thi ở trạng thái RUNNABLE - đang tiêu tốn CPU chứ không bị chặn.

#### Nguyên nhân gốc

Khi xử lý các quy tắc, hàm compile\_rules() chèn trực tiếp (inline) các biểu thức điều kiện vào từng cột đầu ra một cách riêng biệt. Việc thiếu sự tái sử dụng logic này khiến kết quả trả về bị phình to, chứa khoảng 42 cấu trúc CASE WHEN và 119 hàm COALESCE.

Cột max\_severity là nguyên nhân lớn nhất gây bùng nổ kích thước mã. Khối lệnh logic điều kiện đã inline biểu thức max\_rank một lần cho mỗi mức độ nghiêm trọng. Do max\_rank vốn tổng hợp vị từ của cả sáu luật, việc áp dụng cho bốn mức nghiêm trọng đã tạo ra bốn bản sao đầy đủ. Hệ quả là kích thước cây logic tăng theo cấp số nhân (nhân

chuỗi) thay vì cấp số cộng, dẫn đến hiện tượng phình to mã nguồn dù logic gốc tương đối đơn giản.

Bản thân cây to không làm chậm việc xử lý dữ liệu (executor). Vấn đề nằm ở tầng lập kế hoạch (planning) trên driver: ForeachBatchSink gọi lại LogicalRDD.fromDataset() ở mỗi micro-batch, và hàm này phải tính toán lại toàn bộ ràng buộc (constraints) của kế hoạch bằng ExpressionSet - so sánh biểu thức theo cấu trúc (semanticEquals), phải duyệt đệ quy toàn bộ cây con cho từng cặp so sánh. Với  $n$  biểu thức, số cặp tăng theo  $O(n^2)$ ; nếu tính cả việc mỗi lần so sánh cũng tốn công duyệt cây, chi phí thực tế có thể lên tới  $O(n^3)$ .

Đây là lý do cây lớn “không làm chậm xử lý dữ liệu” - nó làm nghẹt bộ tối ưu hóa: phần xử lý dữ liệu thật chạy trên bytecode đã biên dịch sẵn (WSCG), chi phí tuyến tính theo số dòng và ở mức thấp; còn phần lập kế hoạch chạy một lần duy nhất mỗi batch, chi phí tăng phi tuyến theo kích thước cây.

Chi phí tính constraints là chi phí CPU/bộ nhớ thuần túy của quá trình phân tích, không phải một request tới tài nguyên có ngưỡng cứng như heap - nên Catalyst không có cơ chế nào để “báo lỗi khi tính quá lâu”.

**Xác nhận bằng đo đạc trực tiếp:** Gọi .constraints() trên kế hoạch này trả ra OOM. Khi tắt constraintPropagation, cùng đoạn mã đó trả về kết quả sau 6 mili-giây. Vì pipeline này không có join và không có filter nào để đẩy xuống - tức là kết quả của bước suy luận ràng buộc không được dùng vào bất kỳ mục đích nào - chứng tỏ đây chính là điểm nghẽn của chương trình.

### Giải pháp

- a. Tắt constraintPropagation - loại bỏ một bước không cần thiết:

.config(“spark.sql.constraintPropagation.enabled”, “false”)

**Cơ chế tác động:** Tính năng này chỉ phục vụ hai mục đích tối ưu: suy diễn bộ lọc (predicate inference) và cắt tỉa nullability - cả hai đều vô dụng ở đây vì pipeline không có join và không có bộ lọc nào để đẩy xuống. Tắt nó nghĩa là loại bỏ hoàn toàn bước sinh + chuẩn hóa + so sánh cấu trúc  $O(n^2)$  hay thậm chí là  $O(n^3)$  ra khỏi pipeline lập kế hoạch - không phải làm nó nhanh hơn, mà là không thực hiện nó nữa.

**Hiệu quả:** Tức thì và triệt để (từ OOM/vô định => 6 ms). Việc tắt nó không mất gì trong pipeline hiện tại, nhưng lại là một giải pháp phụ thuộc vào cấu hình của pipeline: nếu sau này pipeline có thêm lệnh join cần tối ưu qua suy luận ràng buộc, cần đánh giá lại ảnh hưởng của việc bật/tắt tham số này.

- b. Giảm trùng lặp tại biểu thức gốc - giảm kích thước  $n$ :

```
severity_expr = F.element_at(
    F.array(F.lit(None).cast("string"), *[F.lit(s) for s in SEVERITY_ORDER]),
    max_rank + F.lit(1),
)
```

**Cơ chế tác động:** Đây là giải pháp tận gốc, tác động vào chính  $n$  nên nó có lợi bất kể constraint propagation bật hay tắt. Vì chi phí tính constraint tăng theo bậc lũy thừa của  $n$ , việc giảm  $\sim 45\%$  số node sẽ giảm chi phí lớn hơn đáng kể so với tỷ lệ giảm đầu vào. Ngoài ra, cây nhỏ hơn cũng giảm chi phí ở các bước khác không liên quan tới constraint: kích thước bytecode sinh ra, thời gian resolve/analyze và serialize kế hoạch.

Kết quả đo được trên 6 tập luật thực tế cùng thiết lập:

| Chỉ số                | Trước        | Sau                 |
|-----------------------|--------------|---------------------|
| Số CASE WHEN          | 42           | 20                  |
| Số COALESCE           | 119          | 68                  |
| Độ dài chuỗi kế hoạch | 15,230 ký tự | 11,922 ký tự (-21%) |

*Bảng V-1: So sánh hai phiên bản code trước và sau khi tối ưu*

## Bài học rút ra

Không một bài kiểm thử đơn vị nào có thể phát hiện lỗi này. Mọi bài kiểm thử đều gọi `evaluate()` trên một `DataFrame` theo lô rồi kết thúc bằng `.count()` hoặc `.collect()` - và cả hai đều không kích hoạt việc tính constraints. Chỉ `ForeachBatchSink` mới làm điều đó, và chỉ ở chế độ streaming.

Lỗi này vô hình về mặt cấu trúc đối với kiểm thử thành phần.

Ngoài ra, nó bác bỏ một giả định tương chừng an toàn: “Không dùng Python UDF, chỉ sử dụng biểu thức gốc” không đồng nghĩa với nhanh. Việc trùng lặp biểu thức gây tốn chi phí trong Spark Catalyst, hoàn toàn không liên quan tới executor.

## 2. Bộ đếm bản ghi lỗi luôn bằng 0

### Hiện tượng

Mã nguồn ban đầu lọc bản ghi lỗi bằng:

```
.withColumn("_is_valid", F.col("_parsed").isNull())
```

Giả định ban đầu: `from_json` sẽ trả về struct null khi không phân tích được bản tin. Kiểm chứng trên Spark:

| Bản tin Kafka                              | _is_valid cũ | Kết quả      |
|--------------------------------------------|--------------|--------------|
| Hoàn toàn không phải JSON                  | True         | Lọt qua      |
| JSON hợp lệ nhưng thiếu hầu hết các trường | True         | Lọt qua      |
| JSON hợp lệ nhưng sai kiểu count           | True         | Lọt qua      |
| Bản tin rỗng                               | True         | Bị loại đúng |

*Bảng V-2: Kết quả kiểm chứng các bản ghi lỗi*

Ở chế độ PERMISSIVE (mặc định), nếu một bản ghi không khớp với schema, Spark sẽ trả về một struct có tồn tại, nhưng gán null cho từng field bên trong. `isNotNull()` trên đối tượng struct đó trả về True.

Hệ quả là `malformed_count` vĩnh viễn bằng 0, không bản ghi nào bị lọc, và mọi tin rác đều đi thẳng vào bước tiếp theo dưới dạng một dòng toàn null.

Đáng chú ý, thứ duy nhất ngăn `is_detection` mang giá trị NULL đến cột bool không cho phép null của ClickHouse chính là lệnh `coalesce` trong `rules_engine` - một biện pháp phòng vệ hóa ra giữ vai trò then chốt một cách tình cờ.

### Giải pháp

Thay việc hỏi “Struct có null không?” bằng “Các trường bắt buộc có đến nơi không?”:

```
REQUIRED_FIELDS = (“timestamp”, “protocol_type”, “service”, “flag”, “label”)
```

Bản ghi được phân loại thành ba nhóm: hợp lệ/thiếu sót/không phân tích được (xem chương IV mục 6).

Việc phân loại là một phép chiếu được hợp nhất vào cùng giai đoạn với bước phân tích JSON, và hai nhóm lỗi được đếm bởi chính phép `groupBy` mà `clickhouse_writer` vốn đã thực hiện. Số action của Spark mỗi micro-batch thậm chí giảm từ 4 xuống 2.

### Bài học rút ra

Đây là ví dụ điển hình cho việc một cơ chế giám sát có thể tự nó hỏng mà không ai biết. Bộ đếm báo 0 và mọi người tin rằng dữ liệu sạch - trong khi thực tế bộ đếm không hề hoạt động.

Hệ thống sau khi sửa còn bổ sung nhịp tim (heartbeat): khi không có dữ liệu, chương trình vẫn gửi log định kỳ. Trước đó, một luồng nhân rồi và một luồng bị treo trông giống hệt nhau trong log - cả hai đều không trả ra gì cả.

### 3. Tràn bộ nhớ do micro-batch không giới hạn

#### Hiện tượng

java.lang.OutOfMemoryError: Java heap space  
tại luồng stream execution

#### Nguyên nhân

build\_kafka\_stream() không đặt maxOffsetPerTrigger. Theo mặc định, Structured Streaming tiêu thụ toàn bộ offset khả dụng trong một micro-batch duy nhất.

Trên một topic nhân rồi, điều này không được thể hiện. Nhưng khi có tồn đọng (backlog) - do producer chạy trong lúc job đang dừng, hoặc khởi động lại với checkpoint cũ - toàn bộ tồn đọng trở thành một lô duy nhất, được persist() rồi thu về driver qua toPandas().

#### Giải pháp

1. config.MAX\_OFFSET\_PER\_TRIGGER mặc định ở mức 50,000 - gấp khoảng 3,4 lần tốc độ demo 14,880 bản ghi/giây, nên trạng thái ổn định không bao giờ bị giới hạn; chỉ giới hạn khi bắt kịp tồn đọng.
2. write\_batch() chuyển sang persist(MEMORY\_AND\_DISK) để đề phòng lô quá lớn tràn ra đĩa thay vì giết JVM
3. -driver-memory 4g trong mọi lệnh spark-submit

#### Bài học rút ra

Giới hạn kích thước lô không chỉ là biện pháp chống tràn bộ nhớ mà còn là biện pháp kiểm soát độ trễ. Một lô không giới hạn sẽ vượt chu kỳ kích hoạt 1 giây bất kể luật đánh giá nhanh đến đâu - nghĩa là mục tiêu độ trễ vốn dĩ không thể đạt được nếu không kiểm soát được tham số này.

### 4. ClickHouse chặn truy cập mạng của người dùng không mật khẩu

#### Hiện tượng

Code:194. DB::Exception: default: Authentication failed:  
password is incorrect, or there is no user with such name

Điều gây ra lỗi này là triệu chứng không đối xứng:

| Kết nối                                      | Kết quả                                  |
|----------------------------------------------|------------------------------------------|
| docker exec ids-clickhouse clickhouse-client | <b>Hoạt động</b> - đây là kết nối cục bộ |
| clickhouse_writer.py qua cổng 8123           | Code: 194                                |
| Superset, mọi công cụ GUI                    | Code: 194                                |

*Bảng V-3: Kiểm tra các kết nối tới ClickHouse*

Điều này có nghĩa là SHOW DATABASES chạy được, container báo healthy, nhưng không có gì được ghi vào cơ sở dữ liệu.

### Nguyên nhân

Từ phiên bản ClickHouse 25.1, image chính thức vô hiệu hóa truy cập mạng cho người dùng default nếu không đặt một trong các biến CLICKHOUSE\_USER/CLICKHOUSE\_PASSWORD/CLICKHOUSE\_DEFAULT\_ACCESS\_MANAGEMENT, đồng thời sinh một mật khẩu ngẫu nhiên vào /etc/clickhouse-server/users.d/default-password.xml.

### Giải pháp

Đặt CLICKHOUSE\_PASSWORD: ids\_local\_dev trong docker-compose.yml và đồng bộ giá trị mặc định trong config.py.

### Bài học rút ra

Kiểm tra sức khỏe (healthcheck) của hệ thống là kiểm tra “còn sống” chứ không phải “dùng được”. Lệnh wget -spider https://localhost:8123/ping trong docker-compose.yml vẫn báo thành công bất kể client có xác thực được hay không.

## 5. Chuỗi ràng buộc phiên bản

### Hiện tượng

```
java.lang.NoSuchMethodError:
'scala.collection.mutable.WrappedArray scala.Predef$.wrapRefArray(java.lang.Object[])'
```

### Nguyên nhân

Phương thức này trả về WrappedArray trong Scala 2.12 nhưng trả về ArraySeq trong 2.13 - một trường hợp điển hình của việc trộn lẫn thư viện biên dịch cho hai phiên bản Scala khác nhau.

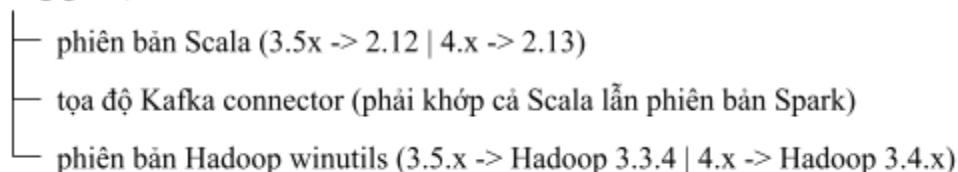


Nguồn gốc là vì pyspark trong requirements.txt không được ghim phiên bản, nên pip install tải về bản 4.x (dùng Scala 2.13), trong khi tọa độ của package Kafka được dùng là spark-sql-kafka-0-10\_2.12:3.5.1 (Scala 2.12).

### Hiệu ứng dây chuyền

Đây là bài học đáng giá nhất về quản lý phụ thuộc trong đồ án: một dòng phụ thuộc không ghim phiên bản đã âm thầm ba quyết định lựa chọn phiên bản khác:

pyspark (không ghim)



Hình V-1: Minh họa ảnh hưởng của pyspark lên các phiên bản ứng dụng khác

Ba lỗi này nằm ở ba tầng khác nhau, và không thông báo lỗi nào chỉ ra nguyên nhân thật của vấn đề.

### Giải pháp

Ghim pyspark == 3.5.1 trong requirements.txt, kèm chú thích giải thích rõ điều gì sẽ hỏng nếu bỏ ghim.

Đồng thời, công cụ tools/check\_env.py được viết để suy ra các giá trị phụ thuộc từ gói đã cài đặt, thay vì đọc từ tài liệu: nó đọc hadoop-client-api-\*.jar trong thư mục jars của PySpark để xác định phiên bản winutils cần dùng, và sinh ra tọa độ - packages chính xác.

## 6. Các vấn đề khác (tóm tắt)

Các vấn đề dưới đây đã được giải quyết nhưng không mang nhiều giá trị phân tích, nên chỉ được nêu ngắn gọn:

### Môi trường chạy Spark trên Windows

- **PySpark lỗi với Python 3.12+ trên Windows:** Tiến trình worker sập trong createDataFrame(); Python 3.13 còn loại bỏ socketserver.UnixStreamServer mà PySpark sử dụng. Giải pháp: Dùng môi trường ảo Python 3.11.
- **Thiếu winutils.exe/hadoop.dll:** Spark trên Windows cần mã native của Hadoop; checkpoint của Structured Streaming không hoạt động nếu thiếu phần này. Không cần cụm Hadoop, chỉ cần hai tệp. Giải pháp: Cài bộ nhị phân Hadoop 3.3.x khớp với phiên bản Spark, đặt HADOOP\_HOME trỏ tới thư mục chứa bin.
- **PySpark không kèm connector Kafka:** Thư mục jars không có jar Kafka nào. Giải pháp: Khởi chạy qua spark-submit --packages.

- **Topic Kafka phải tồn tại trước:** Tự động tạo topic chỉ kích hoạt khi có producer ghi; subscribe của Spark báo lỗi nếu topic chưa tồn tại. Giải pháp: Tạo topic tương minh với số phân vùng xác định.

### Lỗi trong mã nguồn

- **Trường label mang kiểu numpy.str\_:** Tất cả các đặc trưng đã được chuyển sang kiểu Python gốc bằng .item(), riêng label thì không. JSON vẫn tuân tự hóa được (vì numpy.str\_ kế thừa str) nên bộ sinh không hề báo lỗi - nhưng spark.createDataFrame() ném PickleException. Giải pháp: str(label\_arr[i]).
- **Producer không kiểm tra kết quả gửi:** produce() là bất đồng bộ; không có callback nghĩa là worker báo “đã gửi 300,000 bản ghi” mà không biết có bản ghi nào tới nơi hay không. Giải pháp: Thêm callback báo cáo, kiểm tra broker khi khởi động và đảm bảo flush() chạy trong khối finally để không mất dữ liệu khi nhấn Ctrl + C.
- **Lỗi đường dẫn trong validate\_rules.py:** Path(\_\_file\_\_).parent trong khi tệp nằm trong rules khiến chương trình tìm rules/producer/. Giải pháp: parent.parent.

### Cấu hình Superset

- **Thiếu driver ClickHouse trong container:** Image Apache Superset chuyển sang dùng uv sau bản 4.1, nên môi trường ảo không có pip. Giải pháp: dùng mẫu chính thức ./app/.venv/bin activate && uv pip install.
- **Superset chuyển đổi lại cột thời gian về UTC:** Gắn múi giờ trong SQL là chưa đủ; Superset đánh dấu kết quả là cột thời gian rồi chuẩn hóa về UTC. Giải pháp: trả về chuỗi đã định dạng, đồng thời giữ một cột DateTime thật cho bộ lọc.
- **ORDER BY và LIMIT không thuộc về virtual dataset:** Superset bọc SQL thành truy vấn con và thêm mệnh đề của riêng nó; ORDER BY bên trong có thể bị loại bỏ trong khi LIMIT bên trong vẫn được áp dụng - dẫn tới việc chọn ngẫu nhiên một tệp con. Giải pháp: Giữ virtual dataset là phép chiếu thuần túy, đưa khóa sắp xếp ra thành cột, và thiết lập thứ tự ở phần cấu hình biểu đồ.

## VI. Phân tích hiệu năng và mục tiêu độ trễ dưới 2 giây

Mục tiêu “phát hiện trong dưới 2 giây” được đặt ra trong đề cương, trước khi có bất kỳ số đo nào. Sau khi đo đạc, chương này trình bày điều gì thực sự giới hạn hệ thống, đó là lỗi của mã nguồn hay kiến trúc, và các hệ thống thực sự đạt được độ trễ dưới giây được xây dựng khác biệt ra sao.

### 1. Định vị kiến trúc - mục tiêu đến từ đâu

Mục tiêu dưới 2 giây được kế thừa từ trực giác về IDS nội tuyến - một cảm biến ra quyết định trên gói tin đang truyền - rồi được áp dụng cho một pipeline phân tích luồng vốn có nhiệm vụ lưu trữ, phân tích tương quan và báo cáo.

Điểm đáng chú ý nhất: không có kiến trúc chuẩn nào nhắm tới mốc 2 giây.

| Tầng                    | Ví dụ                                                   | Độ trễ điển hình                |
|-------------------------|---------------------------------------------------------|---------------------------------|
| Cảm biến nội tuyến      | Snort 3/Suricata, trên đường truyền                     | 350 $\mu$ s - vài ms            |
| Phân tích luồng bảo mật | Thiết kế tham chiếu IDS trên ksqlDB của chính Confluent | Cửa sổ 60 giây                  |
| Mục tiêu của đề án này  | –                                                       | 2 giây - không thuộc về bên nào |

*Bảng VI-1: So sánh mục tiêu của đề án với các công nghệ hiện hữu*

Hai giây chậm hơn khoảng 5,700 lần so với một cảm biến thực tế, và nhanh hơn khoảng 30 lần so với thiết kế IDS luồng do chính công ty phát triển Kafka công bố. Con số này không đến từ truyền thống nào cả - nó đến từ trực giác về ý nghĩa của “thời gian thực”, và rơi vào khoảng trống giữa hai kiến trúc.

Điều này định vị lại toàn bộ kết luận: sai lệch nằm ở yêu cầu, không nằm ở sản phẩm.

## 2. Kết quả đo và mô hình chi phí

Thời gian xử lý một micro-batch gần như không phụ thuộc vào kích thước lô:

| Kích thước lô | Thời gian | Trên mỗi bản ghi |
|---------------|-----------|------------------|
| 0 bản ghi     | 19 ms     | –                |
| 4,500         | 1,800 ms  | 400 $\mu$ s      |
| 10,700        | 2,000 ms  | 187 $\mu$ s      |
| 15,000        | 2,225 ms  | 148 $\mu$ s      |

*Bảng VI-2: Kết quả đo tốc độ trên mỗi bản ghi theo kích thước lô*

Khối lượng công việc tăng lên 3.3 lần nhưng thời gian chỉ tăng 1.24 lần, khớp với mô hình tuyến tính trên các điểm khác 0:

$$\text{batch\_ms} = 1,618 + 0.0405 \times N$$

(N = số bản ghi trong lô)

Điều đó chứng tỏ rằng 73% thời gian của một lô 15,000 bản ghi là chi phí cố định, không phải công việc thực.

Mô hình dự đoán thông lượng 6,742 bản ghi/giây với giới hạn 15,000; số đo thực tế là 6,743 bản ghi/giây.

Vì độ trễ  $\sim 1,5$  x thời gian một lô (trung bình chờ nửa lô, cộng một lô xử lý), tồn tại một độ trễ sần khoảng 2,5 giây mà không kích thước lô nào có thể vượt qua.

| Tốc độ duy trì   | Giới hạn lô cần thiết | Thời gian một lô | Độ trễ phát hiện trung bình |
|------------------|-----------------------|------------------|-----------------------------|
| 2,500 bản ghi/s  | $\sim 4,500$          | 1.80 s           | $\sim 2.7$ s                |
| 8,000 bản ghi/s  | $\sim 19,500$         | 2.41 s           | $\sim 3.6$ s                |
| 15,000 bản ghi/s | $\sim 62,000$         | 4.13 s           | $\sim 6.2$ s                |

Bảng VI-3: Thời gian một lô và độ trễ phát hiện trung bình trên tốc độ duy trì

### Chi phí đó nằm ở đâu

Phân tách từ chính công cụ đo của Spark (lô  $\sim 10,700$  bản ghi, 1 phân vùng):

| Công việc | Thao tác                                       | Thời gian     | Tỷ lệ       |
|-----------|------------------------------------------------|---------------|-------------|
| A         | Đọc Kafka => phân tích => luật => cache        | 1,029 ms      | 51%         |
| B         | Đọc cache => toPandas() cho detections         | 410 ms        | 20%         |
| C         | Đọc cache => groupBy => số đếm                 | 83 ms         | 4%          |
| —         | Hai lệnh ghi HTTP vào ClickHouse (ngoài Spark) | $\sim 480$ ms | $\sim 25\%$ |

Bảng VI-4: Phân tách công việc ra theo từng thành phần

### 3. Đây không phải lỗi của mã nguồn

Databricks đã công bố nghiên cứu về chính loại chi phí này. Mức nền của họ là 700-900 ms mỗi micro-batch trên phần cứng cụm đã tối ưu, với thông lượng 100 nghìn đến 1 triệu sự kiện/giây. Chẩn đoán của họ mô tả chính xác tình huống của đề án này: thao tác ghi offset log ở đầu mỗi micro-batch và commit log ở cuối “có thể chiếm phần lớn thời gian xử lý, đặc biệt với các pipeline không trạng thái, một giai đoạn.”

Đó chính xác là bản chất của pipeline này. Số đo của họ cho riêng thành phần đó giảm từ 337 ms => 31 ms khi chuyển sang bất đồng bộ.

Con số 1,618 ms trên một laptop - với checkpoint nằm trên NTFS của Windows qua lớp đệm winutils, cộng hai lệnh ghi HTTP đồng bộ mỗi lô - cao hơn khoảng hai lần so với mức sàn Databricks báo cáo trên cụm. Tỷ lệ này tương xứng với môi trường, không phải dấu hiệu bất thường.

#### 4. Giải pháp chính thức không áp dụng được

Apache Spark 3.4+ đã bổ sung async progress tracking (asyncProgressTracking Enabled) trong bản mã nguồn mở, chính là để loại bỏ chi phí offset/commit log nói trên.

Tính năng này không dùng được ở trong phạm vi đề án này. Nó chỉ hỗ trợ các sink no-op, console, memory và Kafka - không hỗ trợ foreachBatch hay bất kỳ sink tùy chỉnh nào. Việc ghi vào ClickHouse bắt buộc phải dùng foreachBatch.

Đây là phát hiện mang tính cấu trúc quan trọng nhất của chương này: thành phần chi phí lớn nhất có thể loại bỏ được lại không tương thích về mặt kiến trúc với việc dùng ClickHouse làm sink.

(Tính năng này cũng làm suy yếu đảm bảo exactly-once - điều tự nó đã là vấn đề đối với một nhật ký kiểm toán an ninh.)

#### 5. Đây có phải vấn đề về công nghệ không?

Một phần.

| Engine                     | Mô hình                     | Độ trễ                 |
|----------------------------|-----------------------------|------------------------|
| Apache Flink               | Luồng thực sự, từng sự kiện | Thấp nhất              |
| Kafka Streams              | Từng sự kiện, dạng thư viện | Thấp, cao hơn Flink    |
| Spark Structured Streaming | Micro-batch                 | Cao nhất - do thiết kế |

*Bảng VI-5: So sánh mô hình Flink, Kafka và Spark*

Spark xử lý dữ liệu theo từng khối hữu hạn theo thời gian; chi phí mỗi lô phải trả bất kể lô chứa một bản ghi hay một triệu. Flink xử lý từng sự kiện ngay khi đến, nên không có khoản phí cố định tương đương.

Chuyển sang Flink sẽ loại bỏ được độ trễ sàn - nhưng đồng nghĩa với việc viết lại toàn bộ trong một framework có hỗ trợ Python yếu hơn PySpark.

## 6. Đây có phải vấn đề về logic không?

Đúng - và đây mới là điểm quan trọng - trên thực tế các hệ thống IDS không thực hiện việc phát hiện bên trong pipeline dữ liệu.

Snort 3 chạy đánh giá luật nội tuyến trên cảm biến, ở tốc độ gói tin, ngay trên thiết bị, cho kết quả trong dưới một mili-giây. Tầng phía sau nhận luồng sự kiện của cảm biến và thực hiện điều tra, làm giàu và tương quan - công việc cần ngữ cảnh thời gian và kết nối chéo nguồn. Nó không thực hiện việc phát hiện.

Bằng chứng hỗ trợ rất đáng chú ý: thiết kế tham chiếu IDS trên ksqldb do chính Confluent công bố sử dụng của số trượt 60 giây và không hề đưa ra tuyên bố nào về độ trễ dưới giây. Công ty tạo ra Kafka khi thiết kế IDS trên chính nền tảng của mình, cũng chỉ nhắm tới mức một phút.

## 7. Hướng đi chưa được xem xét: phát hiện tại biên

rules\_engine.py chỉ đánh giá các ngưỡng trên đặc trưng - không có join, không cửa sổ, không trạng thái, không ngữ cảnh liên bảng ghi. Mọi luật đều quyết định được từ một bản ghi độc lập. Đây chính là tính chất cho phép chuyển việc phát hiện tới nơi bản ghi được tạo ra.

generate\_traffic.py đã dựng bản ghi bằng numpy vector hóa ở tốc độ ~169,000 bản ghi/giây. Sáu luật tương tự có thể được áp dụng tại đó dưới dạng phép toán mảng boolean, tốn thêm vài micro-giây mỗi bản ghi. Kafka khi đó sẽ mang theo mỗi bản ghi kèm sẵn kết luận phát hiện, còn Spark làm đúng việc của một hệ thống micro-batch: lưu trữ, tổng hợp và dashboard - nơi 2-3 giây là hoàn toàn chấp nhận được.

Độ trễ phát hiện sẽ giảm từ ~ 3.3 giây xuống còn vài micro-giây, vì Spark không còn nằm trên đường phát hiện nữa.

Đây không phải ăn bớt - đây chính xác là cách Snort và Suricata hoạt động. Cái giá phải trả mang tính khái niệm: Spark thôi đóng vai trò bộ phát hiện, và điều đó thay đổi tiền đề của đề án.

## 8. Các phương án và đánh giá

| Phương án                                      | Độ trễ đạt được         | Công sức         | Đánh giá                                  |
|------------------------------------------------|-------------------------|------------------|-------------------------------------------|
| Phát biểu lại SLA theo số đo                   | 3.3 s ở 6,700 bản ghi/s | Không            | Được chọn. Con số có thật và bảo vệ được  |
| Chuyển luật sang producer (phát hiện tại biên) | Vài micro-giây          | ~ nửa ngày       | Đúng về kiến trúc; thay đổi tiền đề đồ án |
| Viết lại sink ClickHouse                       | Ước tính 2.1 - 2.6 s    | ~ 1 ngày         | Xác suất đạt dưới 2s khoảng 30%           |
| Chuyển sang Flink                              | Dưới 1 s                | Viết lại toàn bộ | Đúng engine, sai thời điểm                |
| Tiếp tục tinh chỉnh cấu hình Spark             | Không đổi               | —                | Đã cạn phương án khả thi                  |

Bảng VI-6: Đánh giá các phương án cải tiến đồ án

## 9. Kết luận chương

Pipeline được xây dựng không tồi, và phần cứng không phải giới hạn - mức sử dụng CPU trong các lô chỉ đạt 21-68%, tức phần lớn nhân xử lý ở trạng thái rảnh.

Giới hạn nằm ở chỗ xử lý micro-batch thu một khoản phí cố định khoảng 1.6 giây mỗi lô trong môi trường này, giải pháp chính thức cho thành phần lớn nhất của khoản phí đó không tương thích với sink tùy chỉnh, và bản thân mục tiêu thuộc về một tầng kiến trúc khác với tầng đang được đo.

Theo số đo, hệ thống là một pipeline phân tích an ninh gần thời gian thực đúng nghĩa: ~ 6,700 bản ghi/giây, độ trễ phát hiện trung bình ~ 3.3 giây, Recall 98.5% với tỷ lệ báo động giả là 0.59%.

## VII. Kết quả đạt được

### 1. Độ chính xác phát hiện

Kết quả từ một lần chạy trực tiếp qua toàn bộ pipeline (Kafka => Spark => ClickHouse), chấm điểm bằng evaluate\_accuracy.py trên dữ liệu đã lưu thực tế.

Với quy mô 695,000 bản ghi qua 15 micro-batch (660,250 bình thường, 34,750 tấn công), những con số được trả về là:

| Chỉ số                   | Giá trị                      |
|--------------------------|------------------------------|
| Recall                   | 98.5%                        |
| Precision                | 89.9%                        |
| F1-score                 | 94.0%                        |
| Tỷ lệ báo động giả (FPR) | 0.584%                       |
| Accuracy                 | 99.37% (ngưỡng cơ sở: 95.0%) |

Bảng VII-1: Các chỉ số đánh giá hiệu suất mô hình

Ma trận nhầm lẫn (từ một lần chạy khác, 510,000 bản ghi, có đầy đủ bốn ô):

|                      | Dự đoán: Tấn công | Dự đoán: Bình thường |
|----------------------|-------------------|----------------------|
| Thực tế: Tấn công    | TP - 25,121       | FN - 379             |
| Thực tế: Bình thường | FP - 2,885        | TN - 481,615         |

Bảng VII-2: Ma trận nhầm lẫn (Confusion Matrix) với những con số thật

2. Kết quả theo từng loại tấn công

| Loại tấn công      | Số lượng | Bắt được | Bỏ sót | Recall | Precision | F1     |
|--------------------|----------|----------|--------|--------|-----------|--------|
| brute_force        | 5,838    | 5,838    | 0      | 100.0% | 100.0%    | 100.0% |
| ddos               | 5,838    | 5,838    | 0      | 100.0% | 100.0%    | 100.0% |
| port_scan          | 5,838    | 5,838    | 0      | 100.0% | 100.0%    | 100.0% |
| malware_c2         | 5,838    | 5,838    | 0      | 100.0% | 95.8%     | 97.9%  |
| data_exfiltration  | 5,699    | 5,508    | 191    | 96.6%  | 93.0%     | 94.8%  |
| malicious_download | 5,699    | 5,374    | 325    | 94.3%  | 62.7%     | 75.3%  |

Bảng VII-3: Kết quả theo từng loại tấn công



### Phân tích: Vì sao hai luật cuối lại yếu hơn

Bốn luật đầu đạt Recall 100% vì chúng dựa trên tổ hợp nhiều đặc trưng hành vi (số kết nối + tỷ lệ kết nối + kích thước payload), tạo ra vùng nhận dạng tách biệt rõ khỏi lưu lượng bình thường.

Hai luật còn lại là luật ngưỡng theo dung lượng:

```
data_exfiltration_outbound: src_bytes >= 50000 AND dst_bytes <= 1000 AND  
logged_in = 1
```

```
malicious_download_inbound: dst_bytes >= 65000 AND src_bytes <= 500
```

Chúng nằm trên ranh giới của phân phối lưu lượng bình thường: một lần tải lên tệp lớn hợp lệ, hay một lần tải xuống bản cập nhật trông giống hệt về mặt dung lượng. Hệ quả là chúng vừa bỏ sót (tấn công dưới ngưỡng) vừa báo động giả (lưu lượng bình thường trên ngưỡng).

Đây không phải lỗi cần “sửa cho bằng 100%”, mà là minh họa kinh điển cho đánh đổi precision/recall: hạ ngưỡng sẽ tăng điểm Recall nhưng làm sập Precision, và ngược lại.

Đáng chú ý, Precision 62.7% của malicious\_download chỉ lộ ra nhờ cách tính precision theo arrayJoin(matched\_attack\_type) - tức một bản ghi bị gán nhầm loại tấn công khác vẫn bị tính là sai cho loại đó. Nếu chỉ dùng số nhị phân “có phát hiện hay không”, điểm yếu này sẽ bị che khuất hoàn toàn.

### 3. Tính tái lập của kết quả

Ba lần chạy độc lập, chênh nhau hai bậc về quy mô, cho kết quả thống nhất:

| Lần chạy              | Số bản ghi | Precision | Recall | F1    | FPR    |
|-----------------------|------------|-----------|--------|-------|--------|
| Phát lại tồn đọng     | 1,610,000  | 89.8%     | 98.5%  | 93.9% | 0.591% |
| Trực tiếp (lô 50,000) | 695,000    | 89.9%     | 98.5%  | 94.0% | 0.584% |
| Trực tiếp (lô 15,000) | 1,595,000  | 89.8%     | 98.6%  | 94.0% | 0.590% |

Bảng VII-4: So sánh kết quả giữa các lần chạy độc lập

Sai lệch giữa các lần chạy nằm trong khoảng một phần mười điểm phần trăm.

### 4. Pipeline không làm sai lệch dữ liệu

Đây là kết quả có giá trị riêng, đáng được nêu độc lập.

validate\_rules.py chấm điểm rules.json trực tiếp từ bộ sinh, không qua Kafka, không qua cơ sở dữ liệu. Kết quả của nó khớp với pipeline hoàn chỉnh:

| Chỉ số                          | Ngoại tuyến (20,000 bản ghi) | Pipeline trực tiếp (1,61 triệu) |
|---------------------------------|------------------------------|---------------------------------|
| Tỷ lệ báo động giả              | 0.684%                       | 0.591%                          |
| Recall<br>malicious_download    | 94.0%                        | 94.7%                           |
| Recall<br>data_exfiltration     | 97.0%                        | 96.2%                           |
| Precision<br>malicious_download | 64.1%                        | 62.2%                           |

*Bảng VII-5: So sánh kết quả từ validate\_rules.py và pipeline trực tiếp*

**Ý nghĩa:** Quá trình tuần tự hóa JSON, truyền tải qua Kafka, phân tích bằng from\_json, đánh giá luật bằng biểu thức Spark gốc và ghi vào ClickHouse không gây ra thay đổi nào đo được trong kết quả phát hiện.

Kết quả này đồng thời xác nhận ngược lại rằng bộ chấm điểm ngoại tuyến là công cụ đại diện đáng tin cậy để tinh chỉnh luật, cho phép lập trong 30 giây thay vì phải chạy toàn bộ pipeline.

Ngoài ra tỷ lệ lưu lượng đo được ở đầu ra là chính xác 95.00% bình thường / 5.00% tấn công, trùng khớp với tham số cấu hình của bộ sinh - bằng chứng độc lập cho thấy không có bản ghi nào bị mất hoặc nhân đôi trong quá trình.

## 5. Hiệu quả lưu trữ

89,543 dòng được ghi vào bảng detections - tương đương 5.56% tổng lượng dữ liệu đi vào, sát với mức 5.6% mà thiết kế dự đoán.

94.4% lưu lượng còn lại chỉ được đếm trong traffic\_counts thay vì lưu trữ, và toàn bộ các chỉ số ở chương VII mục 1 vẫn được tính đầy đủ.

## 6. Hiệu năng

| Chỉ số                            | Giá trị đo được        |
|-----------------------------------|------------------------|
| Thông lượng duy trì (đầu-cuối)    | ~ 6,700 bản ghi/giây   |
| Thông lượng sinh dữ liệu (đơn lẻ) | ~ 169,000 bản ghi/giây |
| Độ trễ phát hiện trung bình       | ~ 3.3 giây             |
| Chi phí cố định mỗi micro-batch   | ~ 1,618 ms             |
| Chi phí biên mỗi bản ghi          | ~ 40.5 $\mu$ s         |

Bảng VII-6: Một số chỉ số hiệu năng của pipeline

## 7. Dashboard

Dashboard gồm bảy biểu đồ, mỗi biểu đồ chứa các thông tin khác nhau như tỷ lệ tấn công trên tổng số bản ghi, Precision và Recall theo từng loại tấn công, hệ thống nhận bao nhiêu bản ghi mỗi phút.

| Dashboard Live Intrusion Table |                |                    |                                |             |              |                                               |          |          |                  |
|--------------------------------|----------------|--------------------|--------------------------------|-------------|--------------|-----------------------------------------------|----------|----------|------------------|
| window_start                   | verdict        | actual_label       | detected_as                    | event_count | max_severity | rules_fired                                   | conn_min | conn_max | services_touched |
| 2026-08-25 10:24               | CORRECT        | malicious_download | malicious_download             | 940         | high         | malicious_download_inbound                    | 1        | 19       | 3                |
| 2026-08-25 10:24               | FALSE POSITIVE | normal             | data_exfiltration              | 88          | critical     | data_exfiltration_outbound                    | 1        | 39       | 0                |
| 2026-08-25 10:24               | CORRECT        | ddos               | ddos                           | 1008        | critical     | ddos_flood                                    | 200      | 510      | 15               |
| 2026-08-25 10:24               | CORRECT        | port_scan          | port_scan                      | 1008        | medium       | port_scan_sweep                               | 50       | 199      | 15               |
| 2026-08-25 10:24               | FALSE POSITIVE | normal             | malware_c2                     | 46          | high         | malware_c2_beacon                             | 1        | 8        | 0                |
| 2026-08-25 10:24               | FALSE POSITIVE | normal             | malware_c2, malicious_download | 1           | high         | malware_c2_beacon, malicious_download_inbound | 7        | 7        | 0                |
| 2026-08-25 10:24               | CORRECT        | malware_c2         | malware_c2                     | 1008        | high         | malware_c2_beacon                             | 1        | 4        | 4                |

Hình VII-1: Bảng cảnh báo tổng hợp

Bảng cảnh báo tổng hợp là thành phần trung tâm. Thay vì hiển thị hàng nghìn dòng gần giống hệt nhau, bảng gom theo cửa sổ thời gian, loại tấn công, luật, kết luận - đúng cách một bảng điều khiển IDS thực tế.

Cột kết luận (verdict) có bốn trạng thái mang ý nghĩa như sau:

| Kết luận       | Ý nghĩa                                   |
|----------------|-------------------------------------------|
| CORRECT        | Bắt đúng tấn công, gán đúng loại          |
| WRONG TYPE     | Bắt được đúng nhưng gán sai loại tấn công |
| MISSED         | Không bắt được tấn công                   |
| FALSE POSITIVE | Lưu lượng bình thường bị gán cờ           |

Bảng VII-7: Ý nghĩa của các giá trị thuộc cột verdict

Trạng thái WRONG TYPE tồn tại vì một bản ghi có thể bị kích hoạt bởi luật thuộc loại tấn công khác. Nếu thiếu trạng thái này, những trường hợp đó sẽ bị tính nhầm thành đúng - và đó chính là điều che đi vấn đề Precision của malicious\_download.

## VIII. Kết luận

### 1. Những gì đã đạt được

#### Về hệ thống

Đồ án đã xây dựng hoàn chỉnh một pipeline dữ liệu thời gian thực từ đầu tới cuối: sinh dữ liệu mô phỏng, truyền tải qua Kafka, xử lý bằng Spark Structured Streaming, lưu trữ trong ClickHouse và trực quan hóa bằng Superset. Hệ thống đã chạy thực tế, xử lý hơn 1,6 triệu bản ghi trong một lần chạy.

#### Về kết quả phát hiện

Recall 98.5% với tỷ lệ báo động giả 0.59%, tái lập được qua ba lần chạy độc lập với sai lệch dưới một phần mười điểm phần trăm. Bốn trong sáu loại tấn công đạt Recall 100%.

#### Về thiết kế

Một số quyết định thiết kế đã chứng minh được giá trị bằng số đo:

- Thiết kế lưu trữ hai bảng cho phép lưu chỉ 5.56% dữ liệu mà vẫn tính đầy đủ mọi chỉ số, bao gồm cả tỷ lệ báo động giả vốn cần mẫu số không có trong bảng chính
- Tách bạch tầng phát hiện và tầng điều phối cho phép chấm điểm luật ngoại tuyến trong 30 giây, không cần hạ tầng
- Nguyên tắc không dùng UDF giữ toàn bộ đánh giá luật bên trong Catalyst; được kiểm chứng tự động chứ không chỉ tin vào ý định
- Phân loại bản ghi lỗi thành hai nhóm biến bộ đếm từ thông tin thuần túy thành công cụ chẩn đoán chỉ ra tầng nào đang hỏng

#### Về phương pháp

Kết quả có giá trị nhất có lẽ không phải một con số, mà là việc pipeline không làm sai lệch dữ liệu: bộ chấm điểm ngoại tuyến dự đoán chính xác kết quả của hệ thống hoàn chỉnh. Điều này vừa xác nhận tính đúng đắn của toàn tuyến, vừa cung cấp một công cụ tinh chỉnh luật nhanh gấp nhiều lần.

## 2. Những hạn chế

**Mục tiêu độ trễ dưới 2 giây không đạt được:** Số đo cho thấy độ trễ trung bình ~3,3 giây. Phân tích tại Chương VI chứng minh đây là giới hạn kiến trúc chứ không phải lỗi triển khai: micro-batch thu phí cố định ~ 1.6 giây mỗi lô, giải pháp chính thức của Spark cho thành phần lớn nhất của khoản phí đó không hỗ trợ sink tùy chỉnh, và bản thân mục tiêu 2 giây không thuộc về tầng kiến trúc đang được đo.

**Không thể tương quan theo kẻ tấn công:** Bộ dữ liệu NSL-KDD được thiết kế không chứa trường định danh nào — không có địa chỉ IP nguồn/đích, không có session ID. Hệ quả là hệ thống không thể trả lời câu hỏi “địa chỉ nào đã quét cổng rồi tải tệp độc hại”, chỉ có thể tương quan theo cửa sổ thời gian và chữ ký. Đây là hệ quả trực tiếp của việc chọn bộ dữ liệu chuẩn có nhãn, và là giới hạn do thiết kế chứ không phải do sơ suất.

**Bộ sinh không mô hình hóa tấn công như một phiên:** Mỗi bản ghi tấn công được sinh độc lập với giá trị ngẫu nhiên, thay vì mô phỏng một phiên tấn công có số kết nối tăng dần. Điều này khiến một cuộc brute force xuất hiện dưới dạng hàng nghìn sự kiện rời rạc thay vì một sự kiện có diễn biến.

**Chỉ chạy trên một máy:** Toàn bộ hệ thống chạy trên một laptop. Các số đo hiệu năng phản ánh cấu hình đó, và có dấu hiệu suy giảm do nhiệt khi chạy tải liên tục — chi phí CPU trên mỗi bản ghi tăng từ 26  $\mu$ s lên 56  $\mu$ s sau khoảng một phút.

**Dữ liệu chỉ mang tính mô phỏng:** Recall và Precision đo trên lưu lượng tổng hợp có phân phối do chính đồ án định nghĩa. Đây là phép đo đúng cho phạm vi đồ án, nhưng không thể ngoại suy sang lưu lượng mạng thật.

## 3. Hướng phát triển

### Ngắn hạn

- **Chuyển việc phát hiện sang biên** (chương VI mục 7): Đưa sáu luật vào bộ sinh dưới dạng phép toán numpy vector hóa. Độ trễ phát hiện giảm còn vài micro-giây; Spark chuyển sang đúng vai trò tương quan và lưu trữ. Đây là kiến trúc mà một hệ thống IDS thực tế sẽ dùng.
- **Bổ sung trường định danh vào bộ sinh:** Dùng để mở khóa khả năng liên kết theo kẻ tấn công - đánh đổi là không còn tuân thủ chuẩn NSL-KDD.
- **Mô hình hóa tấn công như một phiên:** Số kết nối tăng dần theo thời gian trong một phiên, thay vì giá trị ngẫu nhiên độc lập.

### Trung hạn

- **Viết lại sink ClickHouse:** Gộp hai lệnh ghi HTTP, ghi từ executor thay vì thu về driver, loại bỏ persist(). Ước tính đưa độ trễ về còn 2.1-2.6 giây.
- **Bổ sung mô hình Random Forest:** Triển khai mô hình ML chạy song song với tập luật, so sánh Recall/Precision giữa hai phương pháp trên cùng bộ dữ liệu.

### **Dài hạn**

- **Ứng dụng Apache Flink cho tầng phát hiện:** Tận dụng mô hình xử lý theo từng sự kiện thay vì micro-batch
- **Kiểm thử với lưu lượng mạng thật:** Thu thập bằng Zeek hoặc Suricata, thay thế cho dữ liệu mô phỏng

### **4. Nhận định cuối**

Đồ án đặt ra mục tiêu xây dựng một hệ thống phát hiện xâm nhập thời gian thực và đo đặc hiệu năng của nó. Hệ thống đã được xây dựng, đã chạy, và đã được đo.

Kết quả đo cho thấy một mục tiêu ban đầu không đạt được - nhưng quan trọng hơn, cho thấy vì sao nó không thể đạt được, với mô hình chi phí định lượng, so sánh với các hệ thống công nghiệp, và một phân tích chỉ ra rằng yêu cầu đó vốn thuộc về một tầng kiến trúc khác.

Trong kỹ thuật dữ liệu, việc đo được giới hạn của hệ thống mình xây dựng và giải thích được nguồn gốc của giới hạn đó có giá trị không kém việc đạt một con số đặt ra từ trước khi có bất kỳ phép đo nào.

## TÀI LIỆU THAM KHẢO

1. Anthropic – *Claude AI*  
<https://www.claude.ai>
2. Databricks – *Latency goes subsecond in Apache Spark Structured Streaming*.  
<https://www.databricks.com/blog/latency-goes-subsecond-apache-spark-structured-streaming>
3. Apache Spark – *Structured Streaming Programming Guide: Performance Tips*.  
<https://spark.apache.org/docs/latest/streaming/performance-tips.html>
4. waitingforcode – *What's new in Apache Spark 3.4.0: Async progress tracking for Structured Streaming*.  
<https://www.waitingforcode.com/apache-spark-structured-streaming/what-new-apache-spark-3.4.0-async-progress-tracking-structured-streaming/read>
5. Onehouse – *Apache Spark Structured Streaming vs Apache Flink vs Apache Kafka Streams: Comparing Stream Processing Engines*.  
<https://www.onehouse.ai/blog/apache-spark-structured-streaming-vs-apache-flink-vs-apache-kafka-streams-comparing-stream-processing-engines>
6. Confluent – *Build an Intrusion Detection System using ksqlDB*.  
<https://www.confluent.io/blog/build-a-intrusion-detection-using-ksqldb/>
7. Stack Overflow Blog – *When the sensor starts thinking: SnortML, agentic AI, and the evolving architecture of intrusion detection*.  
<https://stackoverflow.blog/2026/05/11/when-the-sensor-starts-thinking-snortml-agentic-ai-and-the-evolving-architecture-of-intrusion-detection/>
8. Apache Superset – *Docker builds, images and tags*.  
<https://superset.apache.org/user-docs/6.0.0/installation/docker-builds/>
9. ClickHouse Documentation – *Connect Superset to ClickHouse*.  
<https://clickhouse.com/docs/integrations/superset>
10. NSL-KDD Dataset – Canadian Institute for Cybersecurity, University of New Brunswick.